React HandsOn 4:-

Explain the need and Benefits of component life cycle.

The Component Life Cycle in React refers to the different stages a component goes through from its creation until it is removed from the DOM. Lifecycle methods allow developers to perform specific actions at different stages.

Need

- To initialize data when a component is created.
- To fetch data from APIs.
- To update the UI when state or props change.
- To optimize performance by preventing unnecessary renders.
- To clean up resources such as timers or event listeners before the component is removed.

Benefits

- Efficient Data Handling: Load data only when needed.
- Better Performance: Optimize rendering and avoid unnecessary updates.
- Resource Management: Prevent memory leaks by cleaning up resources.
- Improved User Experience: Keep the UI synchronized with application data.
- Easy Debugging and Maintenance: Lifecycle methods clearly define when specific operations occur.

Identify various life cycle hook methods.

React class components have lifecycle methods grouped into three phases.

A. Mounting Phase (Component is Created)

1. constructor()
    - Initializes state and binds methods.
2. static getDerivedStateFromProps()
    - Updates state based on changes in props before rendering.

3. render()
    - Returns the JSX to display on the screen.
4. componentDidMount()
    - Executes after the component is rendered.
    - Commonly used for:
        - API calls
        - Fetching data
        - Starting timers
        - Adding event listeners

B. Updating Phase (State or Props Change)

1. static getDerivedStateFromProps()
    - Updates state when props change.
2. shouldComponentUpdate()
    - Determines whether the component should re-render.
    - Returns true or false.
3. render()
    - Re-renders the UI.
4. getSnapshotBeforeUpdate()
    - Captures information before the DOM is updated.
5. componentDidUpdate()
    - Executes after the component has updated.
    - Used for:
        - API calls based on updates
        - DOM manipulations
        - Side effects

C. Unmounting Phase (Component is Removed)

1. componentWillUnmount()
    - Executes before the component is destroyed.
    - Used to:
        - Clear timers
        - Remove event listeners
        - Cancel network requests
        - Prevent memory leaks

D. Error Handling Phase

1. static getDerivedStateFromError()
    - Updates state when an error occurs.
2. componentDidCatch()
    - Logs errors and displays fallback UI.

List the sequence of steps in rendering a component

Mounting (Initial Rendering)

1. constructor()
2. getDerivedStateFromProps()
3. render()
4. componentDidMount()

Updating (When Props or State Change)

1. getDerivedStateFromProps()
2. shouldComponentUpdate()
3. render()
4. getSnapshotBeforeUpdate()
5. componentDidUpdate()

Unmounting

1. componentWillUnmount()

Note: In modern React, functional components use the useEffect() Hook instead of class lifecycle methods:

- useEffect(() => {}, []) → Similar to componentDidMount()
- useEffect(() => {}) → Similar to componentDidUpdate()
- useEffect(() => { return () => {}; }, []) → Similar to componentWillUnmount()

Output for REACT Lab